



ITI · PYTHON FRAMEWORK WITH FASTAPI TRACK

Python Framework with FastAPI

Eng. Fatma Elraey*Lecture 2 — Request Validation, File Handling & Structured Errors*

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lecture 2 of 5
Focus	Pydantic request/response models, automatic validation, request bodies, file uploads, environment variables, and structured errors.

Welcome!

This session continues the same Documents API and builds directly on the previous module. The focus is not only making endpoints work, but making their contracts explicit, testable, and maintainable.

Today's Outline

1. Pydantic request and response models
2. Automatic request validation and typed bodies
3. File uploads with UploadFile
4. Environment configuration with python-dotenv
5. HTTPException and structured error responses

01

Pydantic Models: Define the Contract

Let's imagine that an endpoint want to receive data: a client want to make a document

```
{
  "title": "API Guide",
  "content": "...",
  "priority": 2
}
```

Who guarantees that

```
title is there
title is not one character
content is not empty
priority is between 1 and 5
```

if we wrote all the validation by ourselves, it would be something like this:

```
if "title" not in data:
    ...

if len(data["title"]) < 3:
    ...

if len(data["title"]) > 120:
    ...

if data["priority"] < 1:
    ...
```

The code will expand a lot.

Pydantic models turn an informal JSON shape into an explicit contract. FastAPI reads type hints, validates incoming data, generates OpenAPI documentation, and can validate outgoing responses.

Validation boundary: bad input stops before business logic

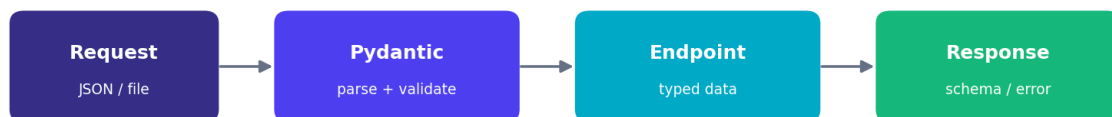


Figure 2.1 — Invalid input stops at the validation boundary before endpoint logic runs.

Request model vs response model

```
from pydantic import BaseModel, Field

class DocumentCreate(BaseModel):
    title: str = Field(
        min_length=3,
        max_length=120
    )

    content: str = Field(
        min_length=1
    )

    priority: int = Field(
        default=1,
        ge=1,
        le=5
    )

class DocumentResponse(BaseModel):
    id: int
    title: str
    content: str
    priority: int
```

- **Request model** — what the client is allowed/required to send.
- **Response model** — what the API promises to return.
- **Field constraints** — lengths, ranges, defaults, and descriptions become validation rules and API documentation.

✓ QUICK CHECK

Q1. Why separate create and response models?

Answer: *The client should not send server-owned fields such as an id.*

Q2. Where does validation happen?

Answer: *Before the endpoint function receives the parsed model.*

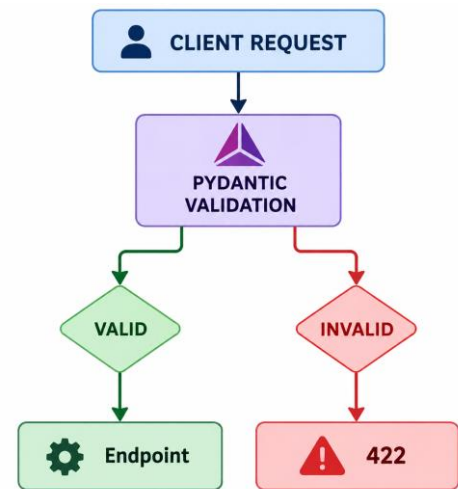
02 Typed Request Bodies & Automatic Validation

Pydantic body in a route

```
from fastapi import FastAPI, status
app = FastAPI()

@app.post(
    "/documents",
    response_model=DocumentResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_document(payload: DocumentCreate):
    new_doc = {
        "id": 105,
        **payload.model_dump(),
    }

    return new_doc
```



to run:

```
python -m uvicorn main:app --reload
```

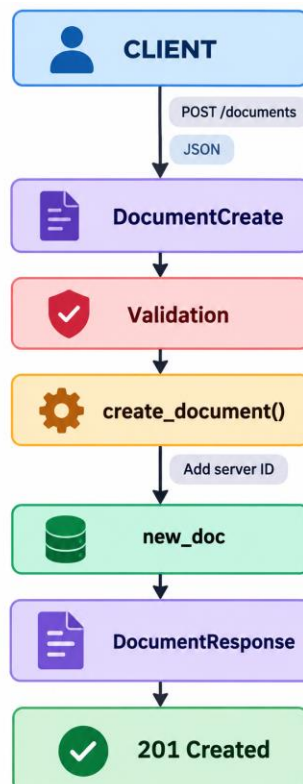
example – validation outcome

VALID {"title": "APIGuide", "content": "...", "priority": 2} → endpoint runs

INVALID {"title": "A", "content": ""} → FastAPI returns 422

IMPORTANT The function receives a DocumentCreate object, not an untrusted raw dict.

PRO TIP Treat Pydantic models as the public boundary of your API. Keep internal database models separate from external request/response contracts.



03 File Uploads with UploadFile

The client want to send a pdf file or txt file so the request will be different not with json.

JSON is suitable for something like this (metadata)

(string, number, boolean, array, object)

```
{
  "title": "API Guide",
  "priority": 2,
  "author": "Omar",
  "is_public": true
}
```

But if you want to send files with your request like:

- An image
- A pdf file
- Video.mp4
- Document.docs
- Audio.mp3

```
title = "My Resume"
description = "Latest version"
file = resume.pdf
```

here we have 2 types of data:

```
Metadata -> (title , description)
+
Binary Data -> (resume.pdf)
```

File uploads use multipart/form-data rather than JSON. FastAPI exposes Uploaded files as UploadFile objects, which provide metadata and a file-like interface without forcing the entire file into memory at once.

Upload endpoint

```
from fastapi import File, UploadFile

@app.post("/documents/upload")
async def upload_document(file: UploadFile = File(...)):
    allowed = {"text/plain", "application/pdf"}
    if file.content_type not in allowed:
        raise HTTPException(
            status_code=400,
            detail="Unsupported file type",
        )

    content = await file.read()
    return {
        "filename": file.filename,
```

```

        "content_type": file.content_type,
        "size": len(content),
    }

```

- **UploadFile.filename** — original client-side filename.
- **UploadFile.content_type** — declared MIME type.
- **await file.read()** — reads uploaded bytes; large files may need streaming or bounded reads in production.

✓ QUICK CHECK

Q1. Why is a file upload not sent as JSON?

Answer: Binary/file form data normally uses multipart/form-data.

Q2. What should be validated besides the filename?

Answer: At minimum type/size rules relevant to the application.

04 Environment Variables with python-dotenv

Configuration such as database URLs, modes, and secrets should not be hardcoded into source files. A `.env` file can be loaded during local development, while production environments normally inject variables directly.

Local environment configuration

```

# .env
APP_NAME = "Documents API"
MAX_UPLOAD_MB = 5
DATABASE_URL = "..."

# settings.py
import os
from dotenv import load_dotenv

load_dotenv()
APP_NAME = os.getenv("APP_NAME", "Documents API")
MAX_UPLOAD_MB = int(os.getenv("MAX_UPLOAD_MB", "5"))

```

PRO TIP Keep `.env` out of version control when it contains secrets. Commit a `.env.example` with variable names, not secret values.

05

Structured Errors

Clients need predictable failures just as much as predictable success responses. HTTPException is the simplest route-level mechanism; custom exception handlers can standardize application-wide error shapes.

Instead of:

```
raise HTTPException(
    status_code=404,
    detail="Document not found"
)
```

Consistent application error schema

```
from fastapi import FastAPI, status, File, UploadFile, Request
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field

app = FastAPI()

# =====
# 1. Pydantic Models
# =====

class DocumentCreate(BaseModel):
    title: str = Field(
        min_length=3,
        max_length=120
    )

    content: str = Field(
        min_length=1
    )

    priority: int = Field(
        default=1,
        ge=1,
        le=5
    )

class DocumentResponse(BaseModel):
    id: int
    title: str
    content: str
    priority: int

# =====
# 2. Custom Application Error
# =====

class AppError(Exception):
    def __init__(
        self,
        code: str,
        message: str,
        status_code: int = 400
    ):
```

```

    ):
        self.code = code
        self.message = message
        self.status_code = status_code

# =====
# 3. Global AppError Handler
# =====

@app.exception_handler(AppError)
async def app_error_handler(
    request: Request,
    exc: AppError
):
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "error": {
                "code": exc.code,
                "message": exc.message,
            },
        },
    )

# =====
# 4. Create Document
# =====

@app.post(
    "/documents",
    response_model=DocumentResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_document(payload: DocumentCreate):
    new_doc = {
        "id": 105,
        **payload.model_dump(),
    }

    return new_doc

# =====
# 5. Upload Document
# =====

@app.post("/documents/upload")
async def upload_document(
    file: UploadFile = File(...)
):
    allowed = {
        "text/plain",
        "application/pdf"
    }

    if file.content_type not in allowed:
        raise AppError(
            code="unsupported_file_type",
            message="Only TXT and PDF files are allowed",

```

```
        status_code=400,  
    )  
  
    content = await file.read()  
  
    return {  
        "filename": file.filename,  
        "content_type": file.content_type,  
        "size": len(content),  
    }
```

Instructor example – success and failure contracts

SUCCESS {"id":105,"title":"API Guide",...}

ERROR {"error":{"code":"document_not_found","message":"Document not found"}}

STATUS Use 404/400/422/500 appropriately; do not hide all failures behind 200.

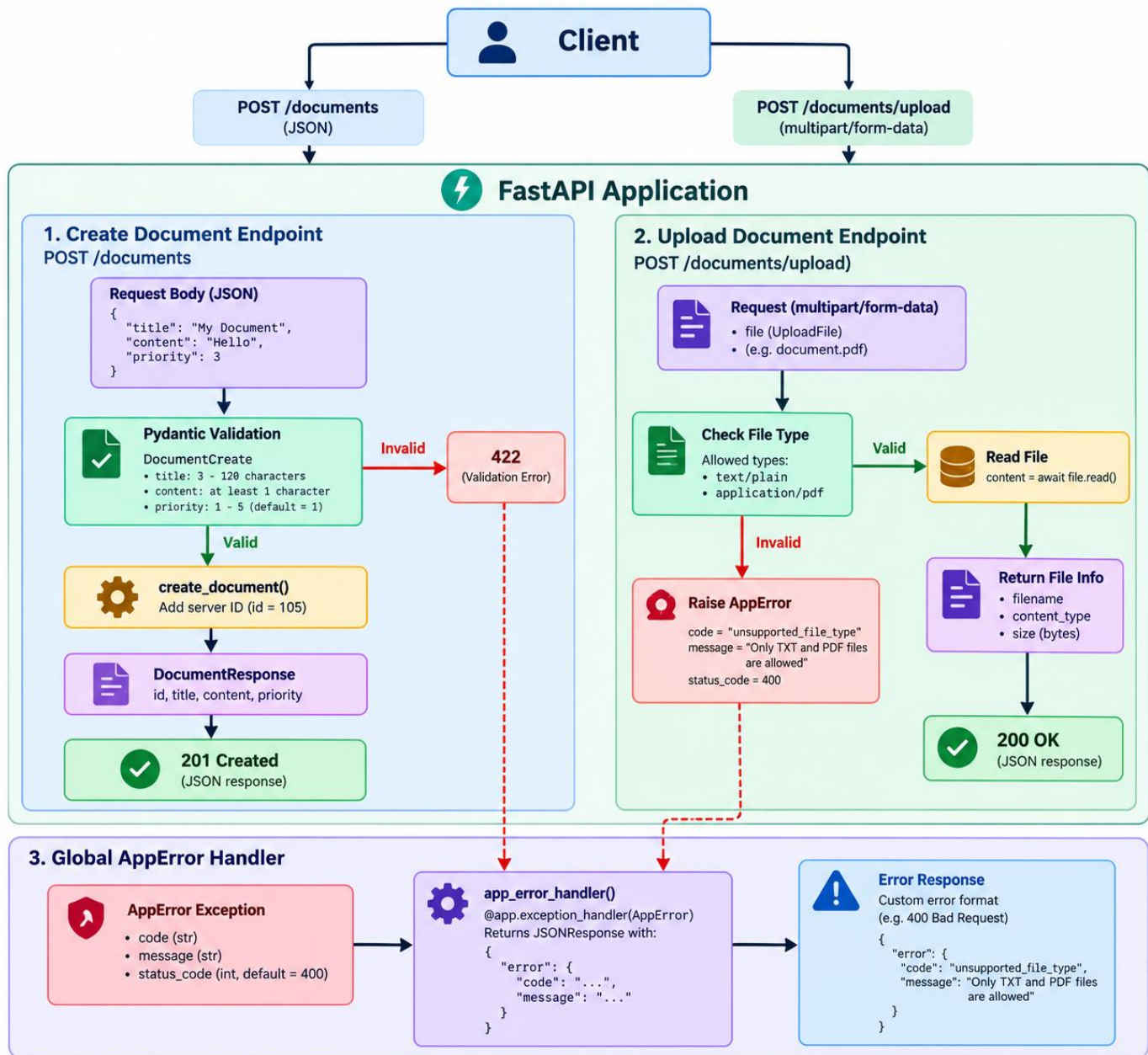
✓ QUICK CHECK

Q1. What does response_model control?

Answer: *The documented and validated response shape.*

Q2. Why use structured errors?

Answer: *Clients can reliably inspect fields instead of parsing arbitrary error text.*



99

Check Your Understanding

Work through these on your own before checking the marked answers.

Practice Question 1

What is the main role of a Pydantic request model?

- A. Render HTML
- B. Validate and parse incoming structured data
- C. Create database tables automatically
- D. Start Uvicorn

Practice Question 2

What status is commonly produced for automatic validation failure?

- A. 201
- B. 404
- C. 422
- D. 301

Practice Question 3

Which FastAPI type is designed for uploaded files?

- A. UploadFile
- B. StateGraph
- C. Session
- D. DependsOnly

Practice Question 4

Where should a local DATABASE_URL normally be stored?

- A. Hardcoded in every route
- B. .env / environment variable
- C. Inside HTML
- D. In the URL path

Practice Question 5

Why define a consistent error schema?

- A. To make clients parse random strings
- B. To make failures predictable for clients
- C. To avoid status codes
- D. To replace Pydantic

Answers

Practice Question 1: B ✓ — It defines and validates the request body contract.

Practice Question 2: C ✓ — FastAPI uses 422 for request validation errors.

Practice Question 3: A ✓ — UploadFile exposes file metadata and async read operations.

Practice Question 4: B ✓ — Configuration belongs outside source code.

Practice Question 5: B ✓ — Structured errors create a stable machine-readable contract.